Prototype

Season: 1 · Creational · Category: GoF · Level: Beginner · Build with Waqar

Today's Hook

- Today: Prototype as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Prototype structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Prototype
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/creational/prototype/DocumentTemplate.java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype: clone a preconfigured object instead of rebuilding from scratch. */
public final class DocumentTemplate implements Cloneable {
    private String title;
    private final List<String> sections;

    public DocumentTemplate(String title, List<String> sections) {
        this.title = title;
        this.sections = new ArrayList<>(sections);
    }

    public String title() { return title; }
    public void setTitle(String title) { this.title = title; }
    public List<String> sections() { return sections; }

    /** Deep copy so mutating the clone does not affect the prototype. */
    @Override
    public DocumentTemplate clone() {
        return new DocumentTemplate(title, sections);
    }
```

Implementation (2)

```
}

/** Modern: records are naturally copyable via canonical constructor / with-style helpers. */
record SlideDeck(String title, List<String> slides) {
    SlideDeck {
        slides = List.copyOf(slides);
    }

    SlideDeck customized(String newTitle) {
        return new SlideDeck(newTitle, slides);
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import java.util.List;

public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Prototype walkthrough ===");
        var proto = new DocumentTemplate("Weekly Sync", List.of("Agenda", "Notes"));
        var teamA = proto.clone();
        teamA.setTitle("Weekly Sync — Team A");
        teamA.sections().add("Action Items");
        System.out.println("protoTitle=" + proto.title() + " sections=" + proto.sections());
        System.out.println("cloneTitle=" + teamA.title() + " sections=" + teamA.sections());
        var deck = new SlideDeck("Kickoff", List.of("Why", "How"));
        var eu = deck.customized("Kickoff — EU");
        System.out.println("deck=" + deck.title() + " eu=" + eu.title() + " sharedSlides=" + (deck.slides() == eu.slides()));
    }
}
```

Walkthrough Output

```
=== Prototype walkthrough ===
protoTitle=Weekly Sync sections=[Agenda, Notes]
cloneTitle=Weekly Sync — Team A sections=[Agenda, Notes, Action Items]
deck=Kickoff eu=Kickoff — EU sharedSlides=true
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Prototype
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Prototype → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.